

Comparativa Técnica: Nuestra Solución vs Engram

Resumen Ejecutivo

engram es un framework de memoria para agentes/LLMs más maduro, genérico y extensible, pensado para construir “memoria persistente” con abstracciones (memorias, stores, retrieval, pipelines), con énfasis en: ingesta indexado recuperación actualización, y APIs reutilizables.

Nuestra solución es una arquitectura “end-to-end” orientada a opencode: integra memoria (STM/LTM/episódica) + razonamiento CoT + herramientas (web/search) + guardrails/fact-checking en un orquestador único. Es fácil de ejecutar y de enseñar, pero menos generalista (menos abstracciones, menos políticas de retención, menos observabilidad y menos “memory lifecycle”).

Conclusión: Nuestra solución encaja perfecto como agente ejecutable en opencode; engram encaja mejor como backend de memoria reutilizable entre múltiples agentes/proyectos. Lo ideal suele ser: usar engram (o ideas de engram) como capa de memoria, y nuestra orquestación (CoT + tools + guardrails) como capa de agente.

Tabla Comparativa

Dimensión	Nuestra Solución (opencode-ready)	engram (Gentleman-Programming/engram)
Propósito	Agente completo (razona + usa herramientas + verifica hechos) con memoria integrada	Framework/lib de memoria para agentes (abstracciones de almacenamiento/recuperación/actualización)
Alcance	End-to-end (orquestador + CLI)	Principalmente capa de memoria (pipeline: ingest embed index retrieve consolidate)

Arquitectura	3 memorias (STM/LTM/episódica) + CoT + Tools + FactChecker + Orchestrator. Simple, monolítica pero clara	Más modular: interfaces/stores/strategies (query rewriting, reranking, decay, dedupe, compaction) más extensible
Persistencia	ChromaDB (persistente en disco) con embeddings sentence-transformers	Probable soporte a múltiples backends (vector DB, KV, SQL) y esquemas de memoria
Recuperación	Búsqueda por similitud (cosine) + concatenación LTM+episódica, sin reranking/rewriting	Probable: query rewriting, hybrid search (vec+BM25), reranking, metadata filtering, time-decay, importancia
Anti-alucinación	FactChecker que extrae claims + busca soporte (LTM/episódica/web) + score de confianza + gate	No es su foco principal. Engram aporta evidencia/recuerdo, no necesariamente “guardrails de verificación”
Razonamiento	CoT con prompt estructurado (plan + respuesta) + inyección de conocimiento/herramientas	Engram típicamente no impone CoT; te da memoria; el agente decide cómo razonar
Herramientas	Tools (web_search con DuckDuckGo, calculator) con integración simple	Engram no prescribe herramientas; es agnóstico
Ciclo de vida memoria	Añadir/buscar/log. Sin consolidación, deduplicación, resumen, expiración, versionado	Esperable: compaction/summarization, conflict resolution, updates, archival, pruning, “memory graph/links”
Observabilidad	Salida JSON rica (plan, retrieved, tool_results, factcheck)	Probable: trazas/logs de retrieval, scores, fuentes, explicabilidad de por qué se recuperó
Producción	Ideal para demo/opencode. Faltan tests, métricas, políticas, manejo de errores robusto	Más orientado a producción si está bien acabado (verificar repo actual: features/estado)

Integración opencode	Listo: un Orchestrator.handle() que puedes llamar desde tu runtime/CLI	Engram sería un servicio/componente dentro de opencode, no un agente completo
---------------------------------	---	--

Mejoras Recomendadas (Inspiradas en engram)

Esquemas de Memoria Tipados

```
from pydantic import BaseModel, Field
from typing import Optional, List, Dict, Any, Literal
from datetime import datetime

MemoryType = Literal["fact", "note", "episode", "task", "tool_output"]

class MemorySource(BaseModel):
    type: Literal["ingest", "user", "assistant", "web", "tool", "system"] = "ingest"
    url: Optional[str] = None
    span: Optional[str] = None
    extraction_method: Optional[str] = None
    doc_id: Optional[str] = None

class MemoryRecord(BaseModel):
    id: str
    type: MemoryType
    text: str
    summary: Optional[str] = None
    entities: List[str] = Field(default_factory=list)
    tags: List[str] = Field(default_factory=list)
    importance: float = 0.5
    confidence: float = 0.9
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    source: MemorySource = Field(default_factory=MemorySource)
    project: Optional[str] = None
    user_id: Optional[str] = None
    metadata: Dict[str, Any] = Field(default_factory=dict)
```